

工程硕士数学 第一次实验报告

Hamming 级数求和

专业： 集成电路工程

班级： 深微硕 6 班

姓名： 陈天乐

学号：

日期： 2026 年 10 月 7 日

实验编号： EXP1

1 实验题目名称

实验一：Hamming 级数求和。计算

$$\psi(x) = \sum_{k=1}^{\infty} \frac{1}{k(k+x)}, \quad x = 0, 0.1, 0.2, \dots, 300, \quad (1)$$

共 3001 个函数值，要求每个结果的**绝对误差**小于 10^{-10} ，并研究如何减少计算步骤和运行时间。输出按 “ x 、 $\psi(x)$ ” 两列排列，函数值保留 12 位小数。

2 实验目的和意义

- 区分截断误差、浮点舍入误差和输出舍入误差，建立能够检验的误差控制准则。
- 认识正项级数的收敛速度，说明 “当前项很小” 并不等于 “剩余总和很小”。
- 利用函数的递推关系共享计算结果，将 3001 次独立求和转化为少量初值计算和线性复杂度的递推。
- 掌握 Euler–Maclaurin 尾项修正以及稳定计算小量差值的方法，体会算法设计比单纯提高硬件速度更重要。
- 使用 C 语言完成程序设计、编译、数值检查和计时，通过独立高精度计算验证全部输出。

本方法的理论意义是将无限求和、解析尾项和稳定递推结合起来；实用价值是在保证精度的同时显著降低计算量。类似思想也适用于具有平移递推关系的特殊函数和数值表生成。

3 计算公式与算法

3.1 直接截断求和及其误差分析

记前 N 项和为

$$S_N(x) = \sum_{k=1}^N \frac{1}{k(k+x)}, \quad R_N(x) = \psi(x) - S_N(x)$$

对 $x \geq 0$, $f_x(t) = 1/[t(t+x)]$ 是正的单调递减函数, 因此由积分比较法可得

$$\int_{N+1}^{\infty} f_x(t) dt \leq R_N(x) \leq \int_N^{\infty} f_x(t) dt \quad (2)$$

当 $x > 0$ 时,

$$\frac{\log(1 + \frac{x}{N+1})}{x} \leq R_N(x) \leq \frac{\log(1 + \frac{x}{N})}{x} \quad (3)$$

当 $x = 0$ 时, 取极限得到

$$\frac{1}{N+1} \leq R_N(0) \leq \frac{1}{N} \quad (4)$$

由于 $R_N(x) \leq R_N(0)$, 统一取 $N = 10^{10}$ 可以将**精确算术**下的截断误差控制在 10^{-10} 附近以下, 但 3001 点总共需要约

$$3001 \times 10^{10} = 3.001 \times 10^{13}$$

次求和项计算, 且尚未控制浮点舍入误差。这就是直接计算速度很慢的主要原因。

3.2 递推链分解

对 $x > 0$ 作部分分式分解:

$$\frac{1}{k(k+x)} = \frac{1}{x} \left(\frac{1}{k} - \frac{1}{k+x} \right).$$

令 $F(x) = x\psi(x)$, 并规定 $F(0) = 0$ 。对收敛的差级数进行望远镜求和, 得到

$$\begin{aligned} F(x+1) - F(x) &= \sum_{k=1}^{\infty} \left(\frac{1}{k+x} - \frac{1}{k+x+1} \right) \\ &= \frac{1}{x+1}. \end{aligned}$$

于是

$$\boxed{\psi(x+1) = \frac{x}{x+1}\psi(x) + \frac{1}{(x+1)^2}.} \quad (5)$$

$\psi(0)$ 单独由 $\pi^2/6$ 计算; 其余点分成 10 条链, 起点分别为

$$0.1, 0.2, \dots, 0.9, 1.0.$$

例如 $0.1, 1.1, \dots, 299.1$ 属于同一条链, $1, 2, \dots, 300$ 属于另一条链。每条链有 300 点, 因此仅需 10 个初值和 $10 \times 299 = 2990$ 次递推。

设 $p_i = \psi(i/10)$, 则适于程序实现的形式为

$$p_i = \frac{i-10}{i} p_{i-10} + \frac{100}{i^2}, \quad i = 11, \dots, 3000. \quad (6)$$

使用整数下标, 而不是反复执行 $x \leftarrow x + 0.1$, 可以避免网格值累积偏移。

3.3 递推稳定性及整体误差控制

若初值误差为 e_0 , 在精确递推下, 由式 (5) 有

$$e_{j+1} = \frac{x_0 + j}{x_0 + j + 1} e_j, \quad e_j = \frac{x_0}{x_0 + j} e_0.$$

因此初值误差随 x 增大而衰减, **正向递推不放大初值误差**; 反向递推则会乘上大于 1 的系数, 不宜使用。

采用 IEEE 754 双精度时, 单位舍入误差 $u = 2^{-53} \approx 1.11 \times 10^{-16}$ 。在本题范围内, 整数 i 和 i^2 可精确表示; 式 (6) 的两个加数均为正, 不存在灾难性相消。令 $\gamma_3 = 3u/(1-3u)$, 每步正数递推的局部舍入误差可按 γ_3 量级估计。每条链最多 299 步, 且 $\psi(x) < 1.65$, 递推舍入的保守量级为

$$1.65 \times 299 \times \gamma_3 < 1.65 \times 10^{-13}.$$

结合式 (9), 若对初值浮点计算另保留 10^{-14} 的预算, 则总误差量级仍小于 2.5×10^{-13} ; 保留 12 位小数带来的绝对舍入误差不超过 5×10^{-13} , 远小于实验限值。

3.4 用 Euler–Maclaurin 公式计算 10 个初值

在 $0 < x \leq 1$ 上, 先直接计算前 N 项, 再对无限尾项作 Euler–Maclaurin 修正。保留 B_2, B_4, B_6, B_8 四个修正项, 有

$$\begin{aligned} \psi(x) = S_N(x) + \int_N^\infty f_x(t) dt - \frac{f_x(N)}{2} \\ - \sum_{r=1}^4 \frac{B_{2r}}{(2r)!} f_x^{(2r-1)}(N) + E_4(x, N), \end{aligned} \quad (7)$$

其中

$$B_2 = \frac{1}{6}, \quad B_4 = -\frac{1}{30}, \quad B_6 = \frac{1}{42}, \quad B_8 = -\frac{1}{30}.$$

由

$$f_x^{(j)}(t) = \frac{(-1)^j j!}{x} (t^{-j-1} - (t+x)^{-j-1})$$

和

$$\int_N^\infty f_x(t) dt = \frac{\log(1+x/N)}{x},$$

可得实际使用的近似式

$$A_N(x) = \sum_{k=1}^N \frac{1}{k(k+x)} + \frac{\log(1+x/N)}{x} - \frac{1}{2N(N+x)} + \sum_{r=1}^4 \frac{B_{2r}}{2r x} (N^{-2r} - (N+x)^{-2r}). \quad (8)$$

本实验统一选择 $N = 20$ 。公式仅用于 $0 < x \leq 1$ 的 10 个初值， $x = 0$ 不代入含 $1/x$ 的表达式。

3.5 初值截断误差的严格上界

Euler–Maclaurin 余项的周期 Bernoulli 函数表示给出

$$|E_4(x, N)| \leq \frac{|B_8|}{8!} \int_N^\infty |f_x^{(8)}(t)| dt.$$

因为 $f_x^{(8)}(t) > 0$ ，且 $f_x^{(7)}(\infty) = 0$ ，所以

$$\begin{aligned} |E_4(x, N)| &\leq \frac{|B_8|}{8x} (N^{-8} - (N+x)^{-8}) \\ &\leq |B_8| N^{-9} = \frac{1}{30N^9}. \end{aligned}$$

第二步使用中值定理 $N^{-8} - (N+x)^{-8} \leq 8xN^{-9}$ 。代入 $N = 20$ ，得到统一上界

$$|E_4(x, 20)| \leq \frac{1}{30 \cdot 20^9} = 6.5104166667 \times 10^{-14}. \quad (9)$$

该上界适用于全部 10 个初值，明显小于 10^{-10} ，为舍入误差和输出舍入保留了充足余量。

3.6 避免尾项计算中的相消

在式 (8) 中，直接计算 $\log(1+x/N)$ 或两个接近的幂的差，会损失部分有效数字。令 $L = \log(1+x/N)$ ，则

$$N^{-2r} - (N+x)^{-2r} = N^{-2r} [-\expm1(-2rL)]. \quad (10)$$

程序用 $\log1p(x/N)$ 计算 L ，用 $\expm1$ 计算 $e^z - 1$ ，而不先形成 $1+x/N$ 或 e^z 再相减。

4 结构程序设计

4.1 程序模块与流程

模块	功能
base_value	前 20 项直接求和、解析积分尾项及四个 Bernoulli 修正项，生成初值。
compute_all	计算 $\psi(0)$ 和 10 个初值，分 10 条链完成 2990 次递推。
direct_sum	普通正向直接求和，仅用于对照实验。
now_seconds	Windows 下使用高分辨率计时器；其他平台采用标准 C 计时。
main	检查结果、保存 3001 点、显示代表点、进行批量计时和对照试验。

程序主流程为：

1. 设置 $N = 20$ 、修正阶数 4 及网格下标上限 3000，使用 `double`。
2. 计算 $p_0 = \pi^2/6$ 。
3. 对 $r = 1, \dots, 10$ ，令 $x_0 = r/10$ ，用式 (8) 求出 p_r 。
4. 对每个 r ，依次令 $i = r + 10, r + 20, \dots \leq 3000$ ，按式 (6) 更新 p_i 。
5. 检查全部结果是否有限、为正且严格递减；按下标顺序写出文件。
6. 对同一可执行程序进行计时，并由独立高精度程序逐点核验。

4.2 数据结构、输出与复杂度

使用长度 3001 的 `double` 数组保存结果，内存约为 $3001 \times 8 = 24008$ bytes，即 23.45 KiB。算法实际计算 200 个直接求和项、40 个尾项修正和 2990 个递推步骤。设网格点数为 M ，初值直接求和长度为 N ，则计算复杂度为

$$O(10N + M), \quad \text{空间复杂度为 } O(M).$$

若只需顺序输出，可只保存 10 条链的最新值，将工作存储降到常数级；本实验为方便检查和计时，保留全部数组。

程序生成以下文件：

- `hamming_results.txt`: 3001 行、两列，函数值保留 12 位小数，与题目输出形式一致。
- `hamming_results.csv`: 除 12 位小数外，另保存 17 位有效数字，供独立校验和数据处理使用。

- `benchmark.txt`: 计时方式、计算次数、批量耗时和直接求和对照数据。

完整可编译 C 程序见附录，同时作为 `hamming.c` 独立保存。

高精度校验使用 Python 3.14 与 `mpmath` 1.3.0，十进制工作精度设为 80 位。独立参考表达式为

$$\psi(x) = \frac{\text{digamma}(1+x) + \gamma}{x} \quad (x > 0), \quad \psi(0) = \zeta(2), \quad (11)$$

其中 γ 是 Euler 常数，`digamma` 是对数 Gamma 函数的导数。

5 结果讨论和分析

5.1 代表性计算结果

3001 个结果全部已保存，下面列出代表点。

x	$\psi(x)$	x	$\psi(x)$
0.00	1.644934066848	10.00	0.292896825397
0.10	1.534607244905	20.00	0.179886982857
0.20	1.440878841547	50.00	0.089984106767
0.50	1.227411277760	100.00	0.051873775176
1.00	1.000000000000	150.00	0.037274537258
2.00	0.750000000000	200.00	0.029390154741
5.00	0.456666666667	250.00	0.024402700998
		300.00	0.020942212934

当 $x = m \in \mathbb{N}_+$ 时，可由望远镜求和得到

$$\psi(m) = \frac{H_m}{m}, \quad H_m = \sum_{k=1}^m \frac{1}{k}.$$

因此 $\psi(1) = 1$ 、 $\psi(2) = 3/4$ ，与程序输出一致。全部值为正，且严格递减，这也与

$$\psi'(x) = -\sum_{k=1}^{\infty} \frac{1}{k(k+x)^2} < 0$$

相符。特别地， $x = 300$ 的结果是 0.020942212934，而并非直接用粗略的 $\log(x)/x$ 代替。

5.2 全部 3001 点的精度核验

以式 (11) 的 80 位计算结果为参考，分别比较 C 内部双精度值和实际写出的 12 位小数值，得到：

校验项目	数值	位置或说明
初值解析截断误差统一上界	6.51042×10^{-14}	$0 < x \leq 1$
10 个初值的最大实际解析截断误差	3.56980×10^{-16}	$x = 0.1$
全部内部值的最大绝对误差	5.23537×10^{-16}	$x = 0.9$
12 位小数输出的最大绝对误差	4.99948×10^{-13}	$x = 160.8$
满足绝对误差小于 10^{-10} 的点数	3001	全部通过

这里的“实际解析截断误差”是以 80 位算术计算 $A_{20}(x)$ 再与参考值比较，因而不与 C 程序舍入误差混淆。内部值的最大误差约为限值的 $1/191000$ ；最终文本输出的最大误差约为限值的 $1/200$ 。

5.3 直接求和对照试验

取 $x = 0$ ，直接用 double 从 $k = 1$ 向上累加；误差仍使用 80 位参考值计算。实际结果如下。

截断项数 N	直接和（约 15 位小数）	实际绝对误差	单点耗时（秒）
10^3	1.643934566681562	9.99500×10^{-4}	7.64080×10^{-7}
10^4	1.644834071848065	9.99950×10^{-5}	7.63094×10^{-6}
10^5	1.644924066898242	9.99995×10^{-6}	7.65704×10^{-5}
10^6	1.644933066848770	9.99999×10^{-7}	7.72392×10^{-4}
10^8	1.644934057834575	9.01365×10^{-9}	7.59942×10^{-2}
2×10^8	1.644934057834575	9.01365×10^{-9}	1.53569×10^{-1}

前四行符合 $R_N(0) \approx 1/N$ 。最后两行直接和完全相同，尽管计算量增加一倍。

其原因是，当部分和约为 1.64 时，相邻双精度浮点数的间隔约为 2.22×10^{-16} 。一项 $1/k^2$ 小于半个间隔后，加到部分和上会被舍掉，发生于

$$k \gtrsim (1.11 \times 10^{-16})^{-1/2} \approx 9.49 \times 10^7.$$

因此，普通从大项到小项的累加不仅速度慢，后期还会停滞，不能靠无限增加 N 来保证 10^{-10} 精度。倒序求和、成对求和或 Kahan 补偿求和可减轻舍入损失，但**不能消除原级数 $1/N$ 级别的截断尾项**，仍需尾项修正或其他加速算法。

5.4 运行时间与计算量比较

加速程序重复计算整张 3001 点表 10000 次，测得总耗时为 0.074388900 秒，平均每张表

$$7.43889 \times 10^{-6} \text{ 秒, 约 } 7.44 \text{ 微秒.}$$

计时采用 Windows 的 QueryPerformanceCounter，不包含文件写入和高精度校验；为减少短时测量误差使用重复批量计时，并以 volatile 读取和累加结果防止无用计算被

优化掉。对照试验前三类及 10^6 项试验也采用重复计时， 10^8 与 2×10^8 项各测一次。计时反映本机本次运行，不作为跨机器的固定性能指标。

方法	计算量/每张表	精度与特点
直接截断到 10^6 项	约 3.001×10^9 项	截断误差约 10^{-6} ，不达标。
直接截断到 10^{10} 项	约 3.001×10^{13} 项	仅在精确算术下控制尾项；普通双精度累加会停滞。
尾项修正与正向递推	200 项、40 个修正项、2990 步	3001 点实际输出误差均小于 10^{-10} 。

按本机 $N = 10^6$ 的单点计时外推，3001 点独立直接求和需约 2.32 秒，而加速方法计算整表约 7.44 微秒。该约 3.1×10^5 的比值只是操作量相近时的估计，并非实测整表直接求和的加速比；且对照的 10^6 项方法尚未达标，不能将两者视为同精度算法。真正重要的是加速方法既降低了计算量，又具有明确的尾项误差界。

5.5 参数、算法特点及可改进之处

- 1. **初值精度**：初值误差沿递推链乘上 x_0/x ，不会放大，但较小的 x 最接近初值精度，因此应优先控制 10 个初值的误差。
- 2. **截断长度**：四个修正项的统一余项界为 $1/(30N^9)$ 。例如 $N = 10$ 时为 3.33×10^{-11} ，理论上已接近达标；选择 $N = 20$ 可显著增加舍入余量，而只多计算 100 个求和项。
- 3. **递推方向**：采用正向递推。反向递推会放大误差，且靠近 $x = 0$ 时还需额外处理。
- 4. **准确性与成本**：不必对每个 x 独立调用高精度函数；仅在实验验证阶段用高精度参考值检查程序，不把验证成本计入求值速度。
- 5. **进一步优化**：可直接用 $\psi(1) = 1$ 替代第 10 个初值求和；若只需流式输出，可用 10 个工作值而不是完整数组；也可按目标精度自动选择 N 与修正阶数。
- 6. **应用边界**：本算法利用步长 0.1 网格中相差 1 的点共享递推。若改为无规律的任意采样点，需要重新组织初值与递推链，而不是机械复用当前索引关系。

5.6 程序实现和调试中的问题及处理

- 1. **$x = 0$ 的除零问题**：初值尾项公式含 $1/x$ ，不能直接代入 0。将 $\psi(0) = \pi^2/6$ 单独计算。
- 2. **网格漂移**：用整数索引 i 生成 $i/10$ ，递推也直接使用索引公式，避免连续加 0.1 造成累计偏移。
- 3. **相消及输出误差**：尾项中使用 `log1p/expm1`；既校验 17 位有效数字保存的内部值，也校验最终 12 位小数文本，不以显示小数位数代替误差证明。

4. **计时被优化**：初次重复计时中，直接求和的相同输入调用被优化为重复使用已知结果，使时间读数接近零。改用 `volatile` 输入并保留计时累加结果后，得到随 N 增长而近似线性增加的实际耗时。
5. **大 N 累加停滞**：通过 10^8 、 2×10^8 项对照确认舍入造成停滞。解决办法不是继续增加项数，而是更换为解析尾项修正与稳定递推。

5.7 实验结论

本实验采用“20 项直接求和 + 四个 Euler–Maclaurin 尾项修正 + 10 条正向递推链”计算 Hamming 级数。初值解析截断误差统一不超过 6.51042×10^{-14} ，全网格实际内部最大绝对误差为 5.23537×10^{-16} ，12 位小数输出的最大绝对误差为 4.99948×10^{-13} ，3001 点全部满足 10^{-10} 的精度要求。

直接求和之所以效率低难以计算，原因在于尾项仅按 $1/N$ 的速率衰减，并且 3001 个点有大量重复计算，以及普通浮点累加后期停滞。实验表明，利用解析关系得到递推式、明确控制尾项并选取稳定计算方式，可以同时改善精度与效率。

A 源代码

```
1  /* Hamming series: 3001 values on x = i/10, absolute target 1e-10.
2     C99, double precision, no external libraries beyond the C math library. */
3  #if defined(_MSC_VER)
4  #define _CRT_SECURE_NO_WARNINGS
5  #endif
6  #include <stdio.h>
7  #include <stdlib.h>
8  #include <math.h>
9  #include <time.h>
10 #if defined(_WIN32)
11 #include <windows.h>
12 #endif
13
14 #define LAST_INDEX 3000
15 #define N_BASE 20
16 #define EM_ORDER 4
17 #define FAST_REPEATS 10000
18 #if defined(_MSC_VER)
19 #define NOINLINE __declspec(noinline)
20 #elif defined(__GNUC__) || defined(__clang__)
21 #define NOINLINE __attribute__((noinline))
22 #else
23 #define NOINLINE
24 #endif
25
26 static volatile double benchmark_sink = 0.0;
27 static volatile double direct_benchmark_x = 0.0;
28
29 static double now_seconds(void)
30 {
31     #if defined(_WIN32)
32         LARGE_INTEGER ticks, frequency;
33         QueryPerformanceCounter(&ticks);
34         QueryPerformanceFrequency(&frequency);
35         return (double)ticks.QuadPart/((double)frequency.QuadPart;
36     #else
37         return (double)clock()/((double)CLOCKS_PER_SEC;
38     #endif
39 }
40
41 /* First N terms, followed by an Euler-Maclaurin tail. */
42 static double base_value(double x)
43 {
44     const double B[EM_ORDER] = {1.0/6.0, -1.0/30.0,
45                                   1.0/42.0, -1.0/30.0};
46     const double n = (double)N_BASE;
47     const double l = log1p(x / n);
48     double sum = 0.0, correction = 0.0;
49     int k, r;
50     for (k = 1; k <= N_BASE; ++k)
51         sum += 1.0 / ((double)k * ((double)k + x));
52     for (r = 1; r <= EM_ORDER; ++r) {
```

```

53     double diff = pow(n, -2.0*r) * (-expm1(-2.0*r*1));
54     correction += B[r-1] * diff / (2.0*r*x);
55 }
56 return sum + log1p(x/n)/x - 1.0/(2.0*n*(n+x))
57     + correction;
58 }
59
60 /* Ten residue classes: exact integer indices avoid drifting x += 0.1. */
61 static NOINLINE void compute_all(double values[LAST_INDEX + 1])
62 {
63     const double pi = acos(-1.0);
64     int r, i;
65     values[0] = pi*pi/6.0;
66     for (r = 1; r <= 10; ++r) {
67         values[r] = base_value((double)r/10.0);
68         for (i = r+10; i <= LAST_INDEX; i += 10)
69             values[i] = ((double)(i-10)/(double)i)*values[i-10]
70                 + 100.0/((double)i*(double)i);
71     }
72 }
73
74 static NOINLINE double direct_sum(double x, int n)
75 {
76     double sum = 0.0;
77     int k;
78     for (k = 1; k <= n; ++k)
79         sum += 1.0/((double)k*((double)k+x));
80     return sum;
81 }
82
83 int main(void)
84 {
85     double values[LAST_INDEX + 1];
86     const int selected[] = {0,1,2,5,10,20,50,100,200,500,
87                             1000,1500,2000,2500,3000};
88     const int ns[] = {1000,10000,100000,1000000,100000000,2000000000};
89     FILE *txt, *csv, *bench;
90     double begin, end;
91     double seconds;
92     int i, j, repeat;
93     compute_all(values);
94     for (i = 0; i <= LAST_INDEX; ++i) {
95         if (!isfinite(values[i]) || values[i] <= 0.0 ||
96             (i > 0 && values[i] >= values[i-1])) {
97             fprintf(stderr, "Invalid or nondecreasing result at index %d\n", i);
98             return EXIT_FAILURE;
99         }
100     }
101     txt = fopen("hamming_results.txt", "w");
102     csv = fopen("hamming_results.csv", "w");
103     bench = fopen("benchmark.txt", "w");
104     if (!txt || !csv || !bench) {
105         fprintf(stderr, "Cannot create output files.\n");
106         if (txt) fclose(txt);

```

```

107     if (csv) fclose(csv);
108     if (bench) fclose(bench);
109     return EXIT_FAILURE;
110 }
111 fprintf(csv, "x,psi_12dp,psi_full_precision\n");
112 for (i = 0; i <= LAST_INDEX; ++i) {
113     fprintf(txt, "%6.2f %.12f\n", (double)i/10.0, values[i]);
114     fprintf(csv, "%.1f,%.12f,%.17g\n", (double)i/10.0,
115           values[i], values[i]);
116 }
117 printf("Selected results (all 3001 values saved in TXT and CSV):\n");
118 for (j = 0; j < (int)(sizeof(selected)/sizeof(selected[0])); ++j) {
119     i = selected[j];
120     printf("%6.2f %.12f\n", (double)i/10.0, values[i]);
121 }
122 fprintf(bench, "algorithm=Euler-Maclaurin + forward recurrence\n");
123 #if defined(_WIN32)
124     fprintf(bench, "timer=Windows QueryPerformanceCounter (elapsed time)\n");
125 #else
126     fprintf(bench, "timer=C clock (CPU time)\n");
127 #endif
128 fprintf(bench, "N=%d\nEM_order=%d\nbase_values=10\n", N_BASE, EM_ORDER);
129 fprintf(bench, "base_term_evaluations=%d\nrecurrence_steps=2990\n",
130       10*N_BASE);
131 fprintf(bench, "truncation_bound=%.17g\n", 1.0/(30.0*pow((double)N_BASE,9.0)));
132 begin = now_seconds();
133 for (repeat = 0; repeat < FAST_REPEATS; ++repeat) {
134     compute_all(values);
135     benchmark_sink += values[repeat % (LAST_INDEX+1)];
136 }
137 end = now_seconds();
138 seconds = end-begin;
139 fprintf(bench, "fast_repeats=%d\nfast_batch_seconds=%.9f\n",
140       FAST_REPEATS, seconds);
141 fprintf(bench, "fast_seconds_per_3001_values=%.12g\n",
142       seconds/(double)FAST_REPEATS);
143 fprintf(bench, "direct_x=0\nN,sum_17digits,repeats,total_seconds,seconds_per_value\n");
144 for (j = 0; j < (int)(sizeof(ns)/sizeof(ns[0])); ++j) {
145     int n = ns[j];
146     int repetitions = 100000000/n;
147     if (repetitions < 1) repetitions = 1;
148     double s = direct_sum(0.0, n);
149     begin = now_seconds();
150     for (repeat = 0; repeat < repetitions; ++repeat)
151         /* Volatile input prevents the optimizer hoisting a pure call. */
152         benchmark_sink += direct_sum(direct_benchmark_x, n);
153     end = now_seconds();
154     seconds = end-begin;
155     fprintf(bench, "%d,%.17g,%d,%.9f,%.12g\n", n,s,repetitions,
156           seconds,seconds/(double)repetitions);
157 }
158 i = (fclose(txt) != 0);
159 i |= (fclose(csv) != 0);
160 i |= (fclose(bench) != 0);

```

```
161     if (i) {
162         fprintf(stderr, "Output write failure.\n");
163         return EXIT_FAILURE;
164     }
165     printf("Checks passed: finite, positive and strictly decreasing.\n");
166     printf("Benchmark saved in benchmark.txt (excludes file output).\n");
167     return EXIT_SUCCESS;
168 }
```

B 附带文件清单

- EXP1.tex: 本报告的独立 LaTeX 源文件，程序已内嵌，编译不依赖外部代码文件。
- hamming.c、hamming.exe: 完整 C 源码及本机生成的 Windows 可执行程序。
- hamming_results.txt、hamming_results.csv: 全部 3001 点的正式结果。
- benchmark.txt: 本次运行的原始计时记录。
- verify_results.py: 独立 80 位高精度校验程序；需要 mpmath。
- validation_summary.json 与逐点误差表: 保存在“验证记录”文件夹，记录全部点的参考值和误差。